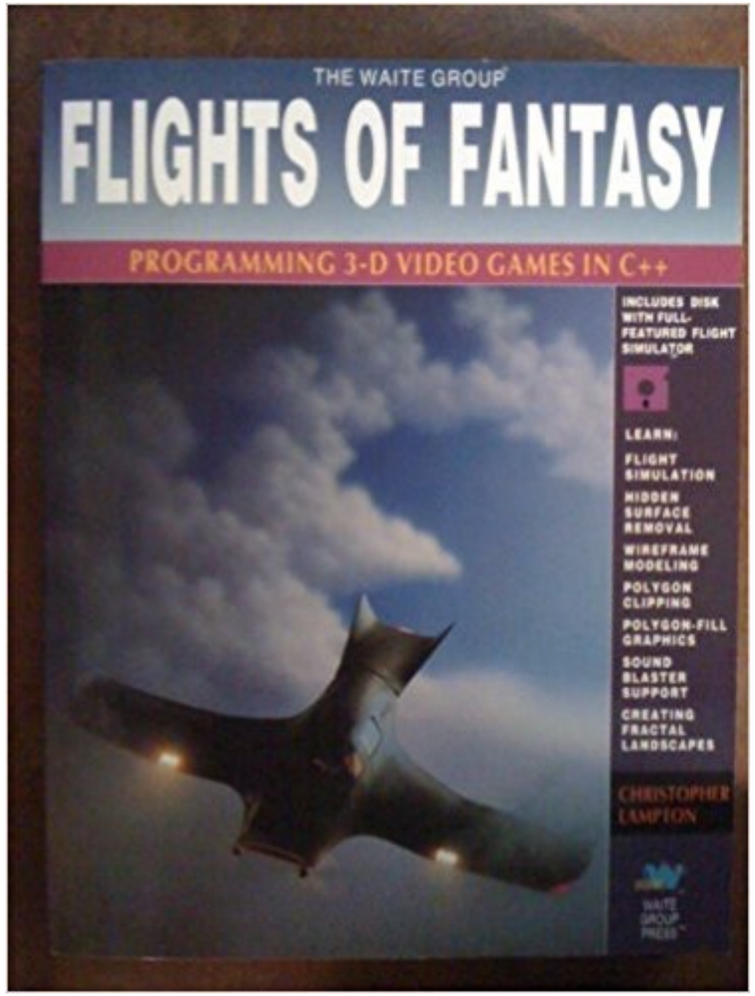
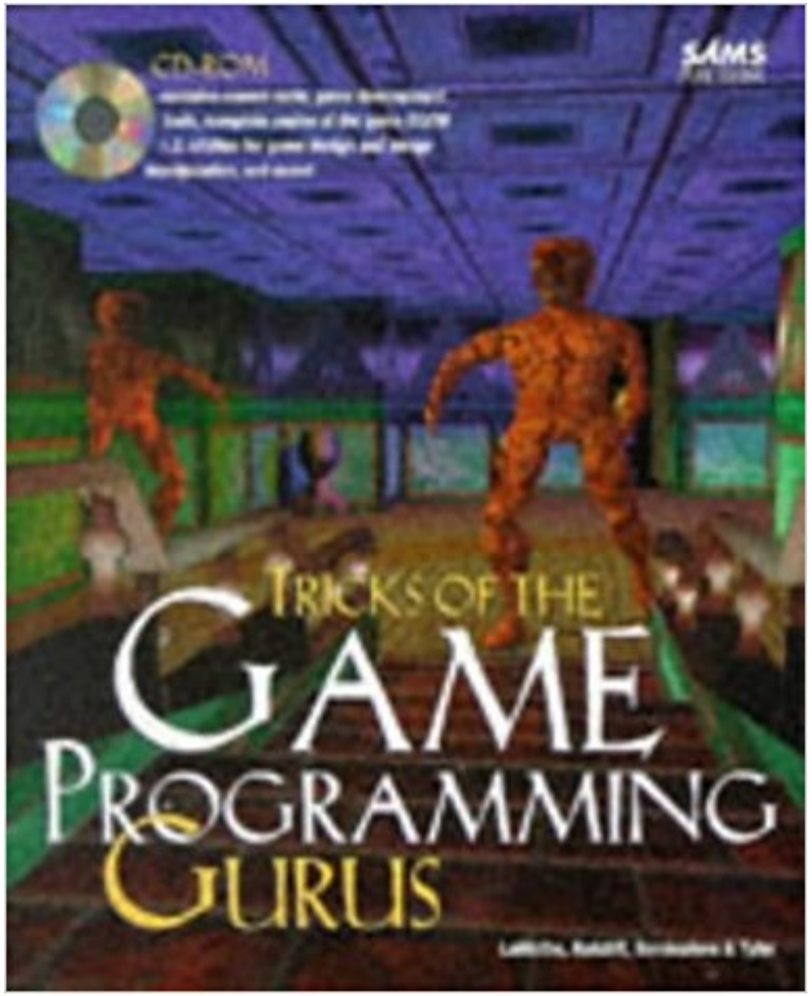
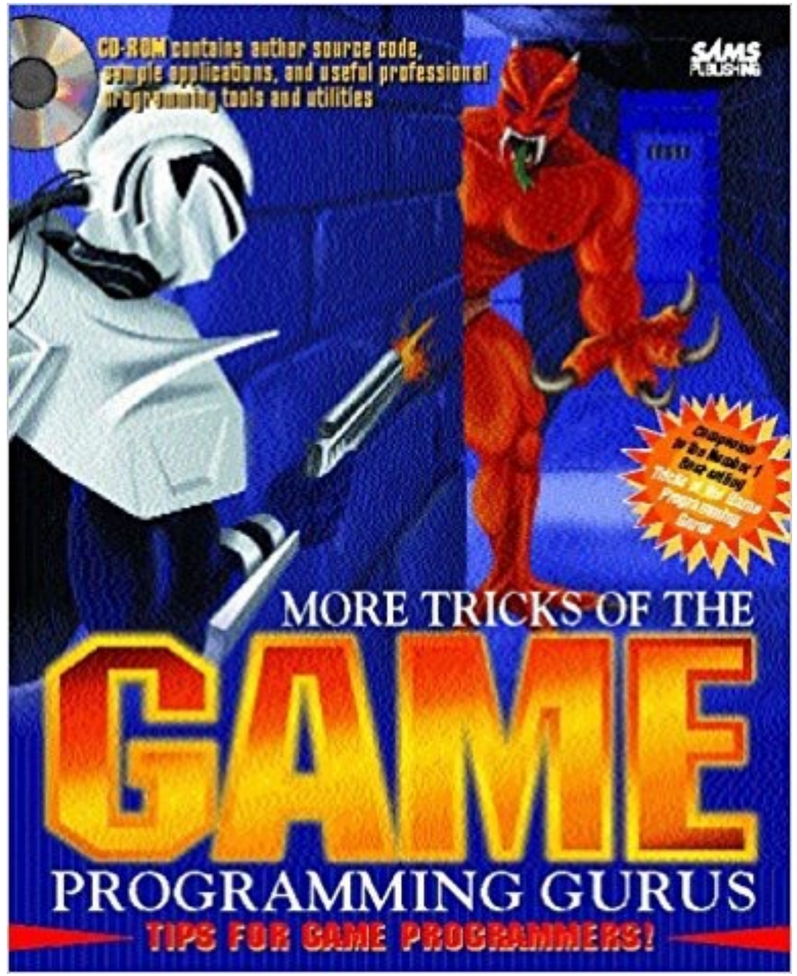
GAME PROGRAMMING
- BASICS -

A. Ricci, DISI - University of Bologna - Italy
a.ricci@unibo.it

SHORT “PERSONAL” PREAMBLE: BACK TO ~1993/1994







NOWADAYS

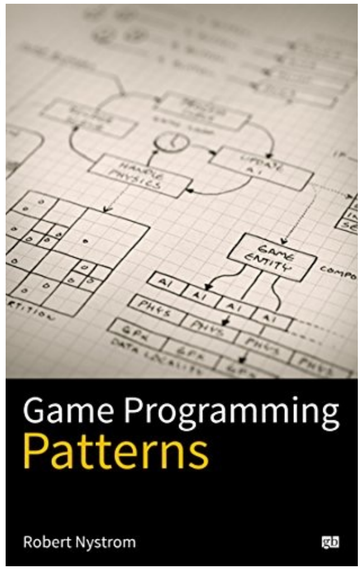
SECOND EDITION
Game Engine Architecture
Jason Gregory
FOREWORD BY Richard Lenk
CRC Press
GAME PROGRAMMING ALL IN ONE
THIRD EDITION
A PLATFORM-AGNOSTIC APPROACH
Game Programming ALGORITHMS and TECHNIQUES
Sanjay MADHAV
GAME AI PRO2
Collected Wisdom of Game AI Professionals
EDITED BY STEVE RABIN
CRC Press
GAME PROGRAMMING Gems 8
Edited by Adam Lake
GAME CODING COMPLETE
FOURTH EDITION
Mike McShaffry and David "Rez" Graham
IN ACTION
Covers Unity 5.0
Joseph Hocking
FOREWORD BY Jesse Schell
MANNING
ARCHITECTING NETWORKED GAMES
MULTIPLAYER GAME Programming
Joshua GLAZER
Sanjay MADHAV
Foundations of Game Engine Development
VOLUME 1
MATHEMATICS
Eric Lengyel
THE ULTIMATE GUIDE TO VIDEO GAME WRITING AND DESIGN
FLINT DILLE & JOHN ZUUR PLATTEN
EDITED BY SKIP PRESS
C++ for GAME PROGRAMMERS
GAME PHYSICS ENGINE DEVELOPMENT
HOW TO BUILD A ROBUST COMMERCIAL-GRADE PHYSICS ENGINE FOR YOUR GAME
SECOND EDITION
Ian Millington
GAME ENGINE DESIGN AND IMPLEMENTATION
Alan Thorn
Beginning Game Programming
Fourth Edition
Jonathan S. Harbour
Second Edition
The Art of Game Design
A Book of Lenses
Jesse Schell
CRC Press
Real-Time Rendering
Third Edition
Tomas Akenine-Möller
Eric Haines
Naty Hoffman

OBJECTIVES

- Introducing some basic game programming aspects, as an application domain where to apply concepts learnt in OOP course (& other bachelor courses)
  - Java as a programming language
  - patterns, software architectures, idioms..
- Method
  - incremental development of a very simple game in Java *("Roll a ball")* in some basic steps, by using some basic game programming patterns

MAIN REFERENCE: GPP

- Reference book
  - **[GPP] Robert Nystrom, “Game Programming Patterns”, 2014**
  - …with some personal adaptations and extensions



GENERAL ASPECTS

- Game architecture: structure and dynamics
- Game world and object modeling
  - structure, behaviour
- Kinematics and dynamics
  - collisions, physics
- Input control & interaction with the physical environment
- Graphics, animation, audio
  - rendering the scene, effects
- AI
- Networking & multi-player

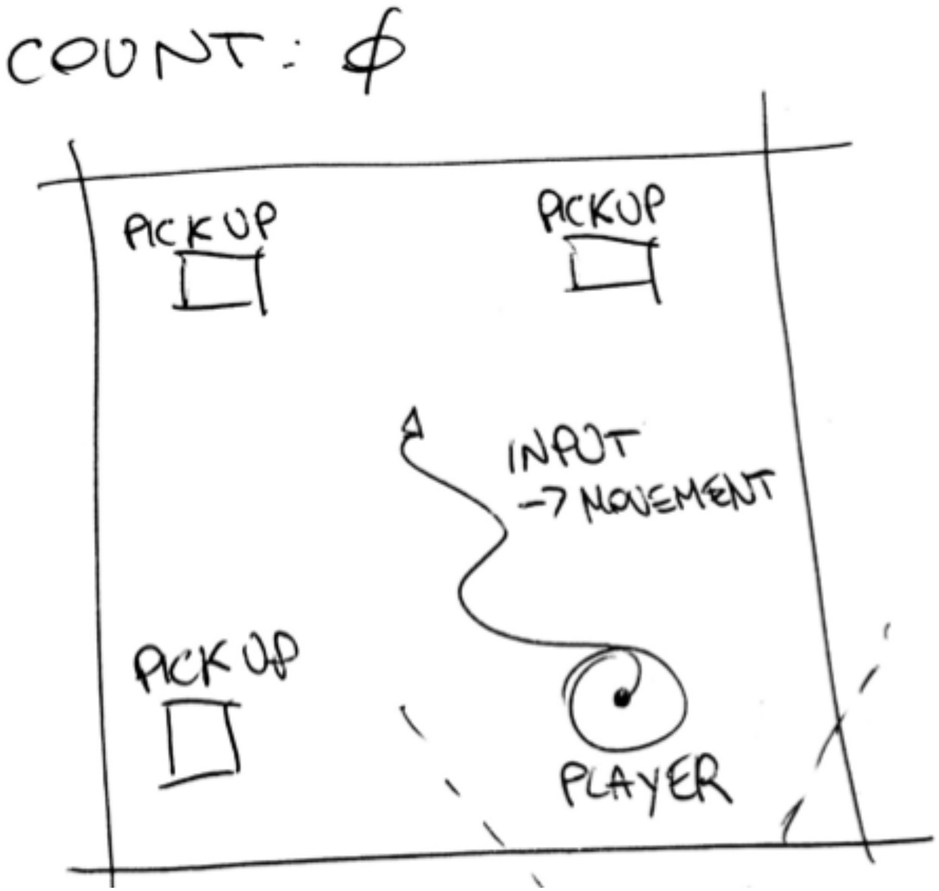
GAME ENGINES

- ***Frameworks*** factorizing functionalities/services
  - ~OS for developing and running games, isolating the game from the HW on which it is running
  - *middleware* perspective
    - vs. libraries
- Many examples
  - http://en.wikipedia.org/wiki/List_of_game_engines

PROGRAMMING A GAME IN FEW STEPS

“ROLL A BALL”

- A *ball* rolling through a 2D environment, moved by the player
- Objective:
  - collect the *pick-up objects* as fast as possible, avoiding borders



(Pierluigi Montagna’s sketch)

PATTERNS IN THIS SHORT JOURNEY

- GAME LOOP
- COMMAND
- OBSERVER
- EVENT QUEUE
- COMPONENT

STEP #0:
MEET THE **GAME LOOP**

THE GAME LOOP

- Control loop ruling the execution of a game
  - *"Decouple the progression of game time from user input and processor speed"* [GPP, CH 9]
- Evolution ~ sequence of frames

```
while (true) {
  processInput()
  update()
  render()
}
```

SYNCH WITH REAL TIME

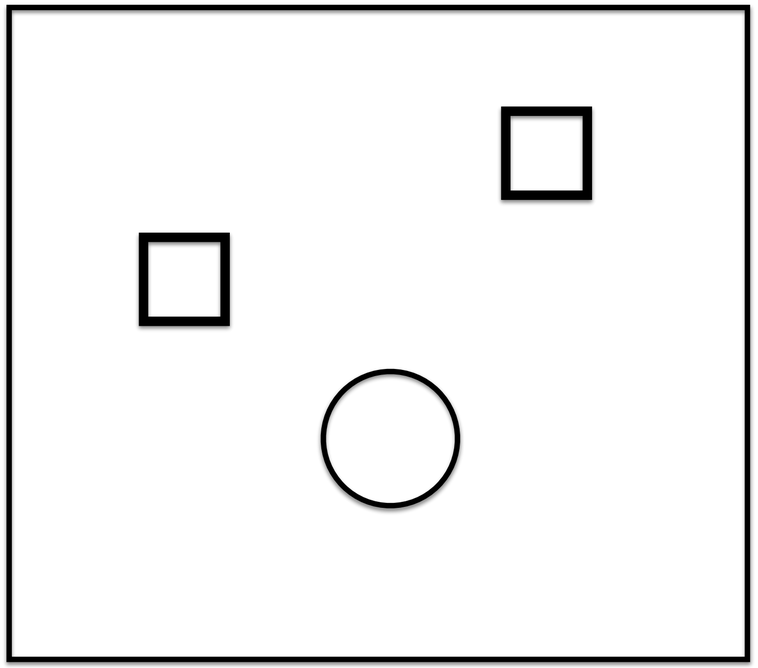
- Choosing a time step to advance the game depending on how much real time elapsed since the previous frame
  - the game runs at the same rate in spite of the specific HW…
  - players with more powerful HW gains in gameplay smoothness
- Check the code: Step #0

```
double lastTime = getCurrentTime()
while (true) {
  double current = getCurrentTime()
  double elapsed = current - lastTime
  processInput()
  update(elapsed)
  render()
  lastTime = current
}
```

STEP #1:
START MODELING THE GAME

START MODELING THE GAME

- OOP modeling of Roll-a-Ball game
  - **GameObject**
    - Ball, PickUpObj
  - **World**
- Basic *separation of concerns*
  - model/graphics
- Check the code: Step #1

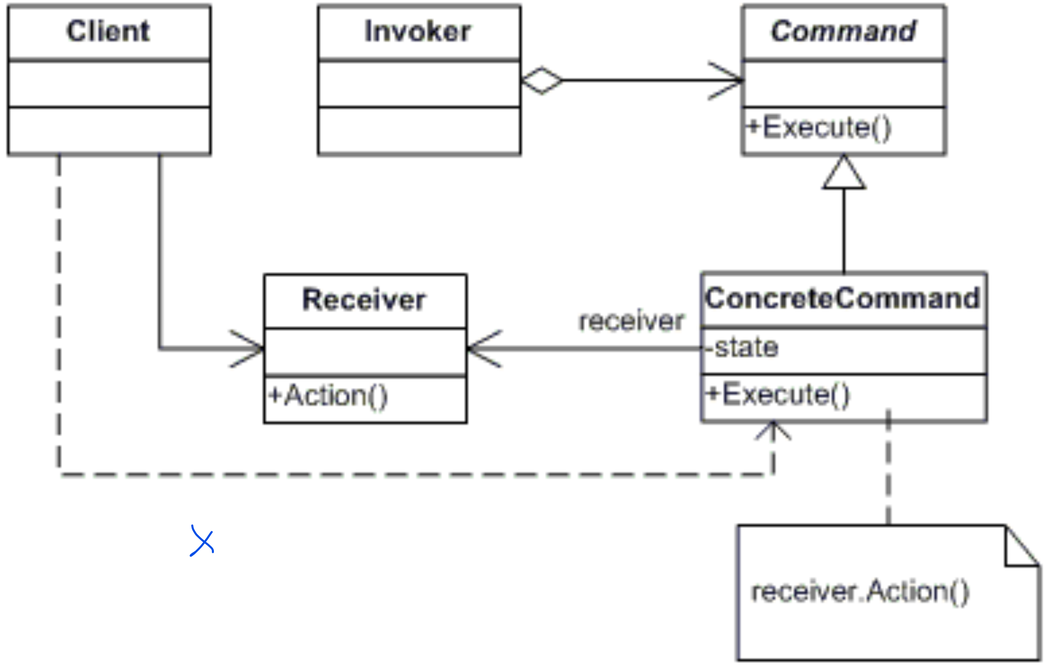


STEP #2:

MEET THE **COMMAND** PATTERN

INPUT PROCESSING

- **Command** pattern
  - *"Encapsulate a request as an object, thereby letting users parametrize clients with different requests, queue or log requests…"* [GPP, CH2]
- Async processing - event listening
  - async keyboard events => commands
- Check the code: Step #2



STEP #3
MEET COLLISIONS

A LITTLE BIT OF PHYSICS: COLLISIONS

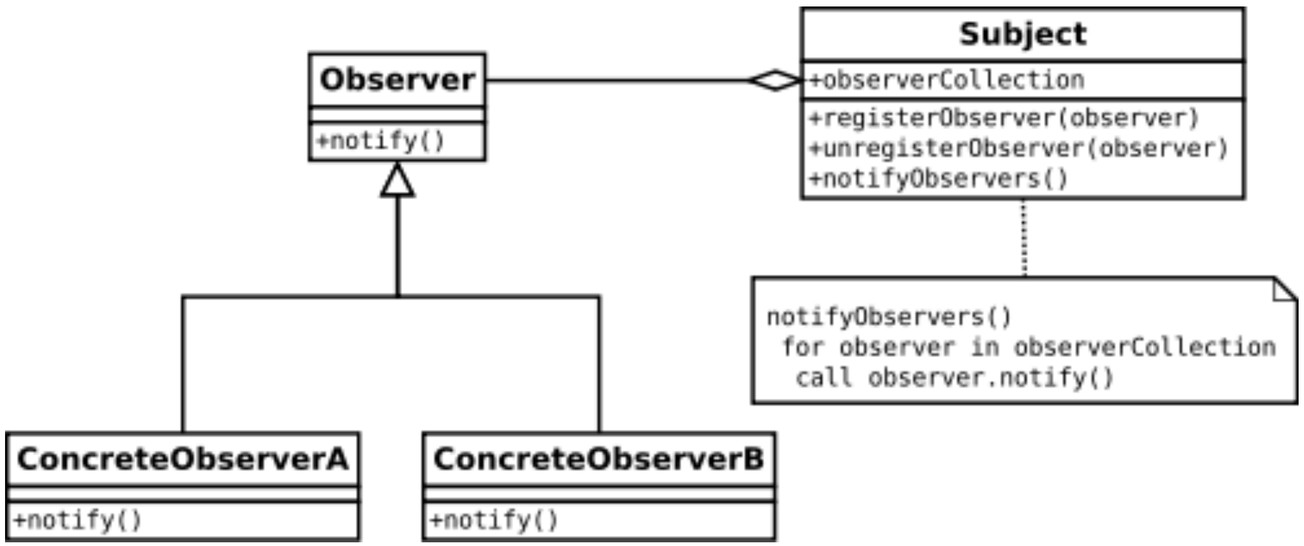
- Simple collision management
  - Ball + PickUpObj, Ball + borders
  - meet Bounding Boxes
- Check the code: Step #3

STEP #4

GAME STATE + EVENTS

GAME STATE + EVENTS

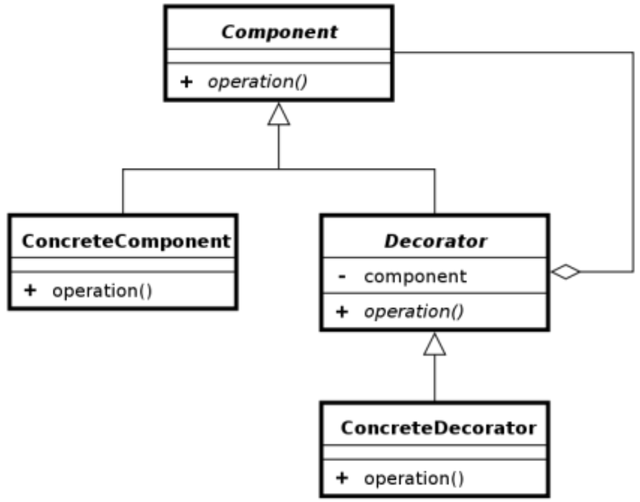
- Introducing the game state, that includes a score
  - score +1 for PickUp, -1 for borders
- Decoupling collision management from the effects
  - **Observer** pattern
    - async variant
- Check the code: Step #4



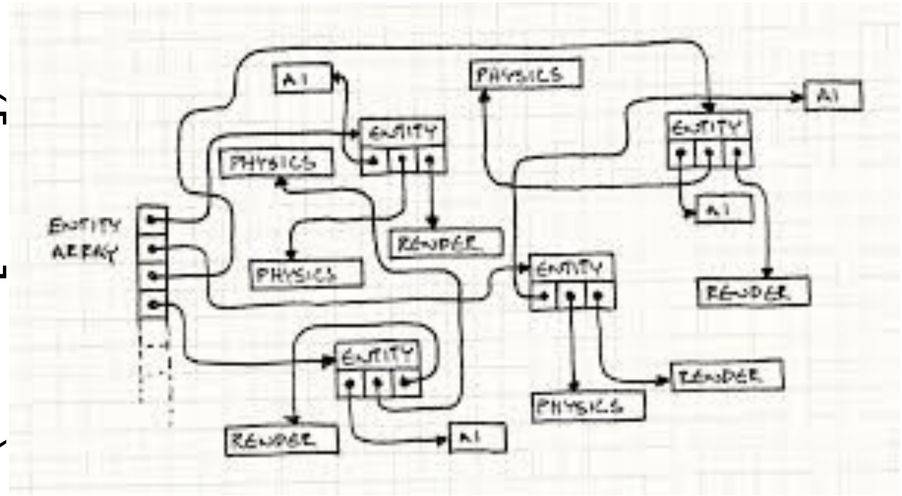
STEP #5
MEET THE **COMPONENT** PATTERN

COMPONENT PATTERN

- **Component** pattern
  - *"Allow a single entity to span multiple domains without coupling the domains each other"* [GPP, CH 14]
- Major refactoring
  - full separation of concerns based on domains (physics, graphics, input)
  - pure Game Objects + Game Object Factory
- Check the code: Step #5



(from [GPP])



STEP #6
GAME OVER

GAME OVER

- Back to the game objective: conditions to end the loop
  - based on GameState
- Game state based rendering
- Check the code: Step #6

GAME OVER

FURTHER STEPS

- Step 7: Mosquito Player
  - first dumb AI player *(MosquitoAIInputComponent)* controlling the ball, random movement
  - development: **State** pattern & AI strategies
- Step 8: Refining physics with accelerations
  - keyboard-based player*(PlayerInputComponentWithAcc)* that produces movements with accelerations
- … *(time exhausted)*

FURTHER STEPS
(NOT DISCUSSED, BUT CODE AVAILABLE)

- Step 9: Multiple Balls
- Step 10: Multi-player
- Step 11: Multi-player with remote users
- Step 12: Multi-player with remote users with view